AvgPool2d#

https://pytorch.org/docs/stable/generated/torch.nn.modules.pooling.AvgPool2d

Description:

Applies a 2D average pooling over an input signal composed of several input planes. In the simplest case, the output value of the layer with input size (N,C,H,W)(N, C, H, W)(N,C,H,W), output (N,C,Hout,Wout)(N, C, H_{out}, W_{out})(N,C,Hout,Wout) and kernel_size (kH,kW)(kH, kW)(kH,kW) can be precisely described as: If padding is non-zero, then the input is implicitly zero-padded on both sides for padding number of points. When ceil_mode=True, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

Function Signature

```
class torch.nn.modules.pooling.AvgPool2d(kernel_size,
stride=None, padding=0, ceil_mode=False,
count_include_pad=True, divisor_override=None)[source]#

Parameters

Shape:

forward(input)[source]#

Return type
```

Returns:

Tensor

Code Examples

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.AvgPool2d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.AvgPool2d((3, 2), stride=(2, 1))
>>> input = torch.randn(20, 16, 50, 32)
>>> output = m(input)
```

Notes & Warnings

> **NOTE** Note When ceil_mode=True, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

> **NOTE** Note pad should be at most half of effective kernel size.

Detailed Documentation

Documentation

Applies a 2D average pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N,C,H,W)(N, C, H, W)(N,C,H,W), output (N,C,Hout,Wout)(N, C, H_{out}, W_{out})(N,C,Hout,Wout) and kernel_size (kH,kW)(kH, kW)(kH,kW) can be precisely described as:

If padding is non-zero, then the input is implicitly zero-padded on both sides for padding number of points.

When ceil_mode=True, sliding windows are allowed to go off-bounds if they start within the left padding or the input. Sliding windows that would start in the right padded region are ignored.

pad should be at most half of effective kernel size.

The parameters kernel_size, stride, padding can either be:

a single int or a single-element tuple – in which case the same value is used for the height and width dimension

a tuple of two ints – in which case, the first int is used for the height dimension, and the second int for the width dimension

kernel_size (Union[int, tuple[int, int]]) – the size of the window

stride (Union[int, tuple[int, int]]) – the stride of the window. Default value is kernel_size

padding (Union[int, tuple[int, int]]) – implicit zero padding to be added on both sides

ceil_mode (bool) – when True, will use ceil instead of floor to compute the output shape

count_include_pad (bool) – when True, will include the zero-padding in the averaging calculation

divisor_override (Optional[int]) – if specified, it will be used as divisor, otherwise size of the pooling region will be used.

Input: (N,C,Hin,Win)(N, C, H_{in}, W_{in})(N,C,Hin,Win) or (C,Hin,Win)(C, H_{in}, W_{in})(C,Hin,Win).

Output: (N,C,Hout,Wout)(N, C, H_{out}, W_{out})(N,C,Hout,Wout) or (C,Hout,Wout)(C, H_{out}, W_{out})(C,Hout,Wout), where

Per the note above, if ceil_mode is True and (Hout−1)×stride[0]≥Hin+padding[0](H_{out} - 1)\times \text{stride}[0]\geq H_{in} + \text{padding}[0](Hout−1)×stride[0]≥Hin+padding[0], we skip the last window as it would start in the bottom padded region, resulting in HoutH_{out}Hout being reduced by one.

The same applies for WoutW_{out}Wout.

Runs the forward pass.